6. Network Scanning using Nmap: Perform scanning of network ports and services to identify vulnerabilities

1. Definition

Network Scanning is the process of examining a computer or network to identify **open ports, active services, and service versions**. It is an important technique used in network administration, security auditing, reconnaissance, and vulnerability assessment.

Nmap (Network Mapper) is an open-source network scanning tool that helps identify available ports and services on a target system.

For this practical, **Google Colab and Python** are used to execute Nmap commands. The target is localhost, which represents the current Colab environment.

> **Security Note:** Network scanning should only be performed on systems that you own or have explicit permission to scan.

2. Objectives

- Understand the concept of network scanning.
- Understand the purpose of Nmap.
- Install Nmap in Google Colab.
- Perform basic port scanning.
- Identify open and closed ports.
- Detect running services.
- Perform service-version detection.
- Execute Nmap commands using Python.
- Analyze Nmap scan results.

3. Learning Outcomes

After completing this practical, students will be able to:

- Explain network scanning.
- Describe the purpose of Nmap.
- Perform port scanning using Nmap.
- Identify different port states.
- Detect services running on ports.
- Use Python to execute Nmap commands.
- Interpret basic Nmap output.

4. Software Requirements

Requirement	Details
Platform	Google Colab
Programming Language	Python 3.x
Network Tool	Nmap
Python Library	subprocess
Target	localhost
Internet	Required for installing Nmap

5. Theory

5.1 What is a Port?

A **port** is a logical communication endpoint used by network applications.

Some commonly known ports are:

Port	Common Service
21	FTP
22	SSH

23	Telnet
25	SMTP
53	DNS
80	HTTP
443	HTTPS
3306	MySQL
8080	HTTP Alternate

5.2 What is an Open Port?

An **open port** means that a service or application is actively listening for network connections.

Example:

22/tcp open ssh

This indicates that TCP port 22 is open and an SSH service has been detected.

5.3 What is a Closed Port?

A **closed port** means that the target system is reachable, but no application is currently listening on that port.

Example:

80/tcp closed http

5.4 What is a Filtered Port?

A **filtered port** means that Nmap cannot clearly determine whether the port is open because a firewall or network filtering mechanism may be blocking the response.

6. What is Nmap?

Nmap stands for Network Mapper.

It is used for:

- Port scanning
- Network discovery
- Service identification
- Version detection
- Security auditing
- Network troubleshooting
- Vulnerability assessment

Basic syntax:

nmap [options] target

Example:

nmap localhost

7. Complete Google Colab Program

Students can **copy the complete program below into one Google Colab cell and execute it directly**.

```
# ============================================================

# PRACTICAL: NETWORK SCANNING USING NMAP

# Environment: Google Colab

# Target: Localhost (Authorized Environment)

# ============================================================



# ------------------------------------------------------------

# STEP 1: Install Nmap
```

```
# ------------------------------------------------------------


import subprocess

import os


print("=" * 70)

print("          NETWORK SCANNING USING NMAP")

print("=" * 70)


print("\n[1] Installing Nmap...")


install = subprocess.run(

    ["apt-get", "update", "-qq"],

    capture_output=True,

    text=True

)


install = subprocess.run(

    ["apt-get", "install", "-y", "nmap"],

    capture_output=True,

    text=True

)


print("Nmap installation completed.")



# ------------------------------------------------------------
```

```
# STEP 2: Check Nmap Version
# ------------------------------------------------------------


print("\n[2] Checking Nmap installation...")


version = subprocess.run(
    ["nmap", "--version"],
    capture_output=True,
    text=True
)


print(version.stdout.splitlines()[0])



# ------------------------------------------------------------
# STEP 3: Define Target
# ------------------------------------------------------------


target = "localhost"


print("\n[3] Target System:", target)



# ------------------------------------------------------------
# STEP 4: Basic Nmap Scan
# ------------------------------------------------------------
```

```
print("\n" + "=" * 70)
print("                    BASIC NMAP SCAN")
print("=" * 70)

basic_scan = subprocess.run(
    ["nmap", target],
    capture_output=True,
    text=True
)

print(basic_scan.stdout)



# ------------------------------------------------------------
# STEP 5: Scan Selected Ports
# ------------------------------------------------------------

print("\n" + "=" * 70)
print("                  SELECTED PORT SCAN")
print("=" * 70)

ports = "22,80,443,8080"

port_scan = subprocess.run(
    ["nmap", "-p", ports, target],
    capture_output=True,
    text=True
```

```
)

print(port_scan.stdout)



# ------------------------------------------------------------
# STEP 6: Service and Version Detection
# ------------------------------------------------------------

print("\n" + "=" * 70)
print("             SERVICE AND VERSION DETECTION")
print("=" * 70)

service_scan = subprocess.run(
    ["nmap", "-sV", target],
    capture_output=True,
    text=True
)

print(service_scan.stdout)



# ------------------------------------------------------------
# STEP 7: Detailed Scan of Selected Ports
# ------------------------------------------------------------

print("\n" + "=" * 70)
```

```
print("       SERVICE DETECTION ON SELECTED PORTS")
print("=" * 70)

detailed_scan = subprocess.run(
    ["nmap", "-sV", "-p", ports, target],
    capture_output=True,
    text=True
)

print(detailed_scan.stdout)



# ------------------------------------------------------------
# STEP 8: Display Errors if Any
# ------------------------------------------------------------

errors = []

if basic_scan.stderr:
    errors.append(basic_scan.stderr)

if port_scan.stderr:
    errors.append(port_scan.stderr)

if service_scan.stderr:
    errors.append(service_scan.stderr)
```

```
if detailed_scan.stderr:
    errors.append(detailed_scan.stderr)


if errors:
    print("\n" + "=" * 70)
    print("                          ERRORS")
    print("=" * 70)

    for error in errors:
        print(error)



# ------------------------------------------------------------
# STEP 9: Completion Message
# ------------------------------------------------------------

print("\n" + "=" * 70)
print("                 NETWORK SCAN COMPLETED")
print("=" * 70)

print("\nTarget Scanned :", target)
print("Ports Checked  :", ports)
print("\nStudents should record the actual Nmap output above.")
```

8. Line-by-Line Program Explanation

Step 1: Program Header

==

PRACTICAL: NETWORK SCANNING USING NMAP

Environment: Google Colab

Target: Localhost (Authorized Environment)

==

These are **comments**.

Comments are not executed by Python. They describe the purpose of the program.

Step 2: Import Libraries

import subprocess

The subprocess module is a built-in Python module.

It allows Python to execute commands provided by the operating system.

In this practical, Python uses subprocess to execute Nmap commands.

import os

The os module provides operating-system-related functionality.

It is included for general system interaction, although the main scanning operations in this program use subprocess.

Step 3: Display Program Title

print("=" * 70)

The print() function displays information on the screen.

"=" * 70

creates a line containing 70 equal signs.

print(" NETWORK SCANNING USING NMAP")

Displays the title of the practical.

print("=" * 70)

Displays another separator line.

9. Installing Nmap

print("\n[1] Installing Nmap...")

Displays the installation status.

\n creates a new line before the message.

Updating the Package List

install = subprocess.run(

["apt-get", "update", "-qq"],

capture_output=True,

text=True

)

This executes the Linux command:

apt-get update -qq

apt-get

apt-get is a Linux package-management command.

update

Updates the package information available to the Linux environment.

-qq

Means **extra quiet mode**, reducing unnecessary installation output.

capture_output=True

Captures the command's output instead of displaying all of it immediately.

text=True

Returns the captured output as normal text.

10. Installing Nmap

install = subprocess.run(

```
    ["apt-get", "install", "-y", "nmap"],
    capture_output=True,
    text=True
)
```

This executes:

apt-get install -y nmap

install

Installs a package.

-y

Automatically confirms the installation prompt.

nmap

Specifies the package that needs to be installed.

Therefore, this command installs **Nmap in the Google Colab Linux environment**.

print("Nmap installation completed.")

Displays a confirmation message.

11. Checking Nmap Installation

print("\n[2] Checking Nmap installation...")

Displays the current operation.

```
version = subprocess.run(
    ["nmap", "--version"],
    capture_output=True,
    text=True
)
```

This executes:

nmap --version

The command checks whether Nmap is installed and retrieves its version information.

---

print(version.stdout.splitlines()[0])

This displays the first line of the Nmap version information.

For example:

Nmap version 7.x

The exact version can vary depending on the Colab environment.

12. Defining the Target

target = "localhost"

This creates a Python variable called target.

Its value is:

localhost

localhost refers to the current machine/runtime.

Using localhost provides a controlled target for this educational practical.

print("\n[3] Target System:", target)

Displays:

[3] Target System: localhost

13. Basic Nmap Scan

```
basic_scan = subprocess.run(
    ["nmap", target],
    capture_output=True,
    text=True
)
```

Python executes:

nmap localhost

This performs a basic Nmap scan.

The result is stored in:

basic_scan

print(basic_scan.stdout)

Displays the output generated by Nmap.

14. Selected Port Scanning

ports = "22,80,443,8080"

Creates a variable containing four selected ports:

22

80

443

8080

These are commonly associated with services such as SSH, HTTP, HTTPS, and alternate HTTP services.

Performing the Scan

```
port_scan = subprocess.run(
    ["nmap", "-p", ports, target],
    capture_output=True,
    text=True
)
```

Python executes a command equivalent to:

nmap -p 22,80,443,8080 localhost

-p

The -p option tells Nmap which ports to scan.

print(port_scan.stdout)

Displays the scan results.

15. Service and Version Detection

```
service_scan = subprocess.run(
    ["nmap", "-sV", target],
    capture_output=True,
    text=True
)
```

This executes:

nmap -sV localhost

-sV

The -sV option enables **service and version detection**.

Nmap attempts to determine:

- Service name
- Service version
- Application information

```
print(service_scan.stdout)
```

Displays the service detection results.

16. Detailed Scan of Selected Ports

```
detailed_scan = subprocess.run(
    ["nmap", "-sV", "-p", ports, target],
    capture_output=True,
    text=True
)
```

This combines:

-sV

and:

-p

The command is equivalent to:

nmap -sV -p 22,80,443,8080 localhost

It scans the selected ports and attempts to identify their services and versions.

print(detailed_scan.stdout)

Displays the detailed scan results.

17. Error Handling

errors = []

Creates an empty Python list.

The list will store error messages if any command produces an error.

```
if basic_scan.stderr:
    errors.append(basic_scan.stderr)
```

Checks whether the basic scan generated an error.

If an error exists, it is added to the errors list.

The same process is performed for the other scans:

```
if port_scan.stderr:
    errors.append(port_scan.stderr)
if service_scan.stderr:
    errors.append(service_scan.stderr)
if detailed_scan.stderr:
    errors.append(detailed_scan.stderr)
```

18. Displaying Errors

if errors:

Checks whether the errors list contains any errors.

If errors exist, the following section is executed.

```
for error in errors:
    print(error)
```

The for loop displays each error message.

19. Completion Message

```
print("=" * 70)
print("                NETWORK SCAN COMPLETED")
print("=" * 70)
```

Displays the final completion message.

```
print("\nTarget Scanned :", target)
```

Displays the target that was scanned.

Example:

Target Scanned : localhost

```
print("Ports Checked  :", ports)
```

Displays the selected ports.

Example:

Ports Checked : 22,80,443,8080

```
print("\nStudents should record the actual Nmap output above.")
```

Instructs students to use their **actual execution results** for the practical observation.

20. Understanding the Nmap Output

A typical output may look like:

```
PORT     STATE    SERVICE
22/tcp   open     ssh
80/tcp   closed   http
443/tcp  closed   https
```

PORT

22/tcp

Means:

- 22 → Port number
- tcp → Transport protocol

STATE

open

Means a service is listening on the port.

closed

Means no service is listening.

filtered

Means filtering or firewall behavior prevents Nmap from determining the state clearly.

SERVICE

ssh

Indicates the service associated with the port.

21. Important Nmap Options Used

Option	Purpose
nmap localhost	Basic scan
-p	Specify ports
-sV	Detect services and versions
--version	Display Nmap version

22. Execution Flow

Start

↓

Import Python Libraries

↓

Install Nmap

↓

Check Nmap Version

↓

Set Target = localhost

↓

Perform Basic Scan

↓

Scan Selected Ports

↓

Detect Services and Versions

↓

Display Results

↓

Check Errors

↓

Display Completion Message

↓

End

23. Algorithm

Algorithm: Network Scanning Using Nmap

1. Start.
2. Import the subprocess module.
3. Install Nmap in Google Colab.
4. Verify the Nmap installation.
5. Set localhost as the target.
6. Perform a basic Nmap scan.
7. Scan selected ports.
8. Perform service and version detection.
9. Display the scanning results.
10. Check for errors.
11. Display the completion message.
12. Stop.

24. Observation Table

Students should fill the table using their **actual Colab output**.

Sr. No.	Port	Protocol	State	Service	Version
1	22	TCP	_____	_____	_____
2	80	TCP	_____	_____	_____
3	443	TCP	_____	_____	_____
4	8080	TCP	_____	_____	_____

26. Result

The practical successfully demonstrated **network scanning using Nmap and Python in Google Colab**. The program installed Nmap, performed port scanning, identified port states, and detected available services and their versions on the authorized local environment.